of, though; there are two optional parameters at the end of the function signature: one is a flag that adds more information to the `$matches` array, and the other is an offset value that starts the matching process at a specified byte in the sample string.

```
$text = 'Cowboy bebop';
preg_match('/\w*bo\w*/', $text, $matches, PREG_OFFSET_CAPTURE, 5);
print_r($matches);
```

```
Output:
    [0] => Array
        (
            [0] => bebop
            [1] => 7
        )
```

The `PREG_OFFSET_CAPTURE` flag adds the byte offset value for the match, and the integer `5` as the final argument tells PCRE to only start matching from the fifth byte in the sample. Thus our matching starts at the `'o'` in `Cowboy`, so we get `'bebop'` as our match and the value `7` as an extra element in the results array because that word begins at the seventh byte in the sample. The other flag supported by `preg_match()` is `PREG_UNMATCHED_AS_NULL`, which sets unmatched capture groups to `null` rather than an empty string:

```
preg_match('/\w*bo\w*(\d)?/', $text, $matches, PREG_UNMATCHED_AS_NULL);
var_dump($matches);
```

```
Output:
array(2) {
  [0]=>
  string(6) "Cowboy"
  [1]=>
  NULL
}
```

Notice that only `'Cowboy'` is matched, even though `'bebop'` is also a match for the pattern. We know by now that if we want global matching behavior, we should call `preg_match_all()`, and this function accepts the same offset and flags as `preg_match()`. It also has additional flags for controlling behavior.